FINAL PROJECT REPORT

Date	2-October- 2026
TeamID	6ab46978b5170a9b58b3f461
ProjectName	**Implement Client Script & UI Policy (Incident)**
Maximum Marks	3 Marks

FINAL PROJECT REPORT
Implement Client Script & UI Policy (Incident)

Team ID: 6ab46978b5170a9b58b3f461
Date: 2-October-2026

Project Team

Member	Role
Nidhila G S	Project Leader
T S Ajith Rajaa	Team Member
Jenish M	Team Member
Joshila D	Team Member
Arshana M	Team Member

Chapter 1: Introduction

Introduction

The **Implement Client Script & UI Policy (Incident)** project is a ServiceNow-based micro project developed to improve the consistency and accuracy of Incident data entry. Incident records may contain incomplete or inconsistent information when users rely only on manual data entry and checking. To address this issue, the project uses **UI Policies, UI Policy Actions, and Client Scripts** to control field behavior and apply validation rules directly on the Incident form. Implement Client Script & UI Po…

The project demonstrates how ServiceNow client-side controls can make fields mandatory, automatically populate values, control field properties, and prevent incorrect record submission. The implementation includes a **High Impact Control UI Policy**, an **Urgency UI Policy Action**, and three Client Scripts: **onChange, onSubmit, and onCellEdit**. Implement Client Script & UI Po…

When an Incident has **Impact = 1 – High**, the configured UI Policy makes **Assignment Group mandatory**, while the Urgency field is controlled through the UI Policy Action. The onChange Client Script automatically sets Urgency to High, the onSubmit Client Script validates Assigned To before saving, and the onCellEdit Client Script prevents direct State modification through list editing. Implement Client Script & UI Po… Implement Client Script & UI Po… Implement Client Script & UI Po…

The project also includes testing activities covering mandatory validation, successful Incident submission, reverse-condition behavior, list-edit blocking, and form-based State updates. These activities demonstrate the practical working of the configured UI Policies and Client Scripts within ServiceNow Incident Management.

FINAL PROJECT REPORT

Chapter 2: Problem Statement and Ideation

Problem Statement and Ideation

1. Problem Statement

Incident records in ServiceNow require consistent and accurate data entry for effective triage, routing, and resolution. However, relying only on user awareness and manual checking can result in incomplete, inconsistent, or incorrect information being submitted. Such issues can affect reporting accuracy, SLA compliance, and overall service quality. Therefore, there is a need to enforce appropriate field behavior and validation directly at the user-interface level. Implement Client Script & UI Po…

The project addresses this problem by implementing **UI Policies and Client Scripts** for the Incident table. The proposed solution applies additional controls when an Incident has **Impact = 1 – High**, including making Assignment Group mandatory and controlling the Urgency field. Client Scripts are used to automate Urgency, validate Assigned To during submission, and restrict State changes through list editing. Implement Client Script & UI Po… Implement Client Script & UI Po… Implement Client Script & UI Po…

2. Ideation

The ideation process identified several requirements for improving Incident form control:

1. **High Impact Control** – Apply a UI Policy when Impact is High.
2. **Assignment Group Validation** – Make Assignment Group mandatory for High Impact Incidents.
3. **Urgency Control** – Make Urgency read-only when the High Impact condition is active.
4. **Automatic Urgency Update** – Use an onChange Client Script to set Urgency to High.
5. **Assigned To Validation** – Use an onSubmit Client Script to prevent submission when Assigned To is empty.
6. **State Update Restriction** – Use an onCellEdit Client Script to prevent State changes through list editing.

These ideas were combined into a single ServiceNow Incident solution that uses UI Policies and Client Scripts to provide controlled, validated, and consistent Incident data entry. Implement Client Script & UI Po… Implement Client Script & UI Po…

Figure 2.1: Problem Statement and Ideation for Implement Client Script & UI Policy (Incident)

Ideation Flow:

Incident Data Inconsistency

↓

Identify Required Controls

FINAL PROJECT REPORT

↓

UI Policy for High Impact

↓

Assignment Group Mandatory + Urgency Control

↓

Client Scripts for Automation and Validation

↓

Controlled Incident Management

Figure 2.2: Brainstoming and Idea Prioritization
Implement Client Script & UI Policy (Incident)

1. Problem Identification

- Incomplete or inconsistent data entry in Incident records
- Missing Assignment Group and incorrect Urgency values
- Impact on reporting accuracy and SLA compliance
- Reliance on user awareness and manual validation

2. Brainstorming of Ideas

- Use UI Policy to control **field behavior**
- Use Client Script to auto-**populate values**
- Make fields mandatory
- Validate records before save
- Restrict direct field editing from list view

3. Evaluation / Prioritization

- Evaluate feasibility
- Assess business impact
- Consider implementation effort
- Identify priority based on importance to data quality and SLA compliance

4. Selected Project Idea

Implement Client Script & UI Policy (Incident)

Use UI Policies and Client Scripts to enforce conditional field behavior and validation on the Incident table.

Idea No.	Proposed Idea	Description	Feasibility (1-5)	Importance (1-5)	Total Score	Priority
1	**UI Policy for High Impact Control**	Make Assignment Group mandatory and control Urgency when Impact is High	**5**	**5**	**10**	**High (Selected)**
2	**Client Script – Auto Set Urgency**	Automatically set Urgency to High when Impact is High	5	5	10	High
3	**Client Script – Validate Assigned To**	Prevent record submission if Assigned To is empty for High Impact	4	5	9	High
4	**Client Script – Restrict State Editing**	Prevent direct State changes from list view (onCellEdit)	4	4	8	Medium
5	**Auto-Populate Additional Fields**	Automatically set other fields based on conditions	3	4	7	Medium
6	**Notification for Missing Information**	Send notification when mandatory fields are missing	3	3	6	Low

5. Final Solution

Implement UI Policies and Client Scripts on the Incident table to ensure complete, consistent, and valid data entry, improving data quality, reporting accuracy, and SLA compliance.

Figure 2.1: Ideation Diagram

Chapter 3: Requirements

- FR-01: The solution shall operate on the ServiceNow Incident table.
- FR-02: An active High Impact Control UI Policy shall be configured with Impact = 1 – High.
- FR-03: Assignment Group shall become mandatory when the High Impact condition is met.
- FR-04: Urgency shall become read-only when Impact is High and the behavior shall reverse when the condition is false.
- FR-05: An onChange Client Script shall automatically set Urgency to High when Impact changes to High.
- FR-06: An onSubmit Client Script shall prevent saving a High Impact Incident when Assigned To is empty.
- FR-07: An onCellEdit Client Script shall prevent direct State changes through list editing.
- FR-08: The configuration shall be tested for mandatory validation, successful save, reverse condition, list-edit blocking, and form-based State update.

FINAL PROJECT REPORT

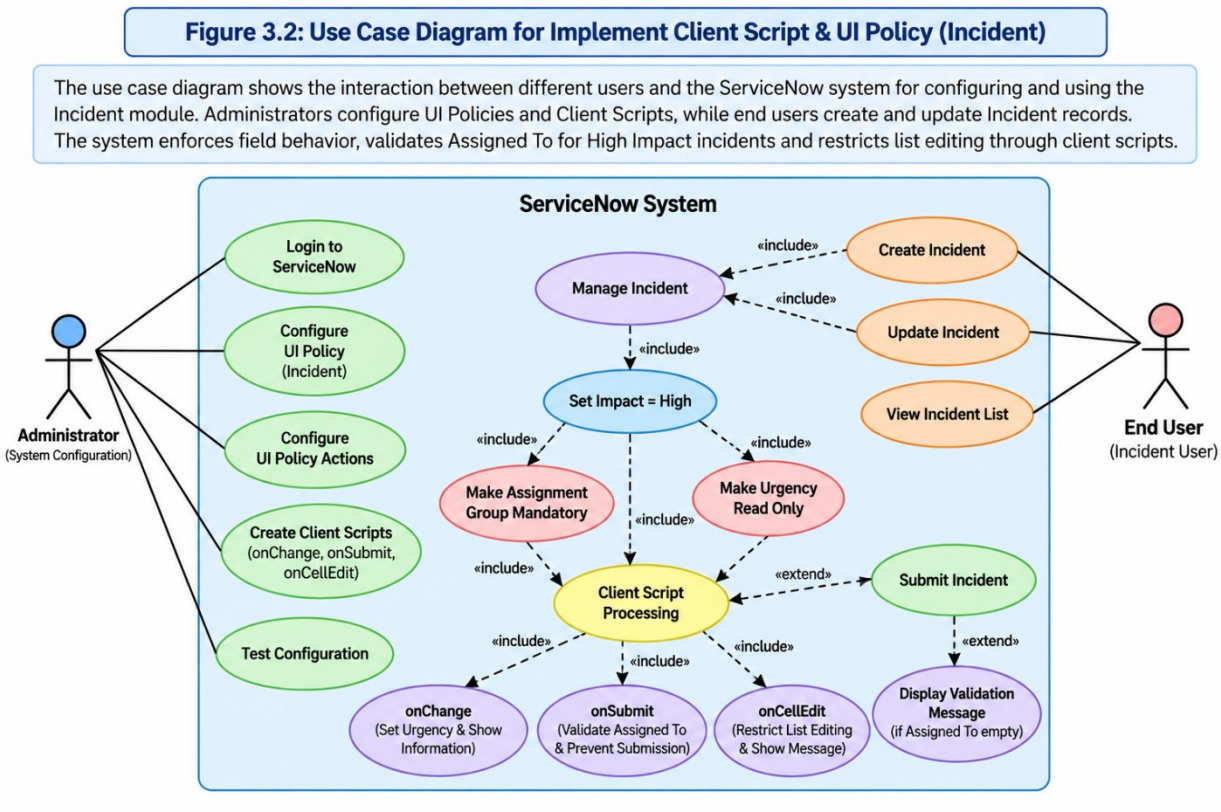


Figure 2.3: Requirement Analysis Flow

Chapter 4: Project Design

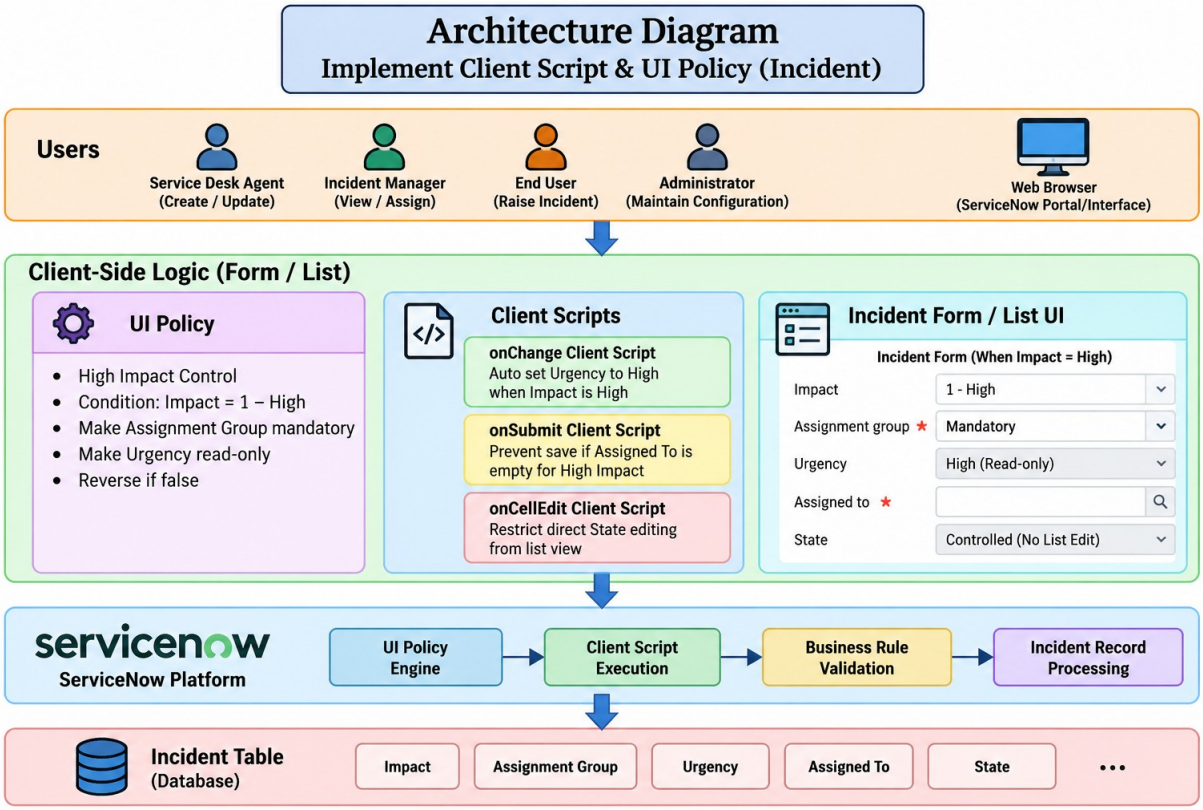


Figure 3.1: System Architecture

FINAL PROJECT REPORT

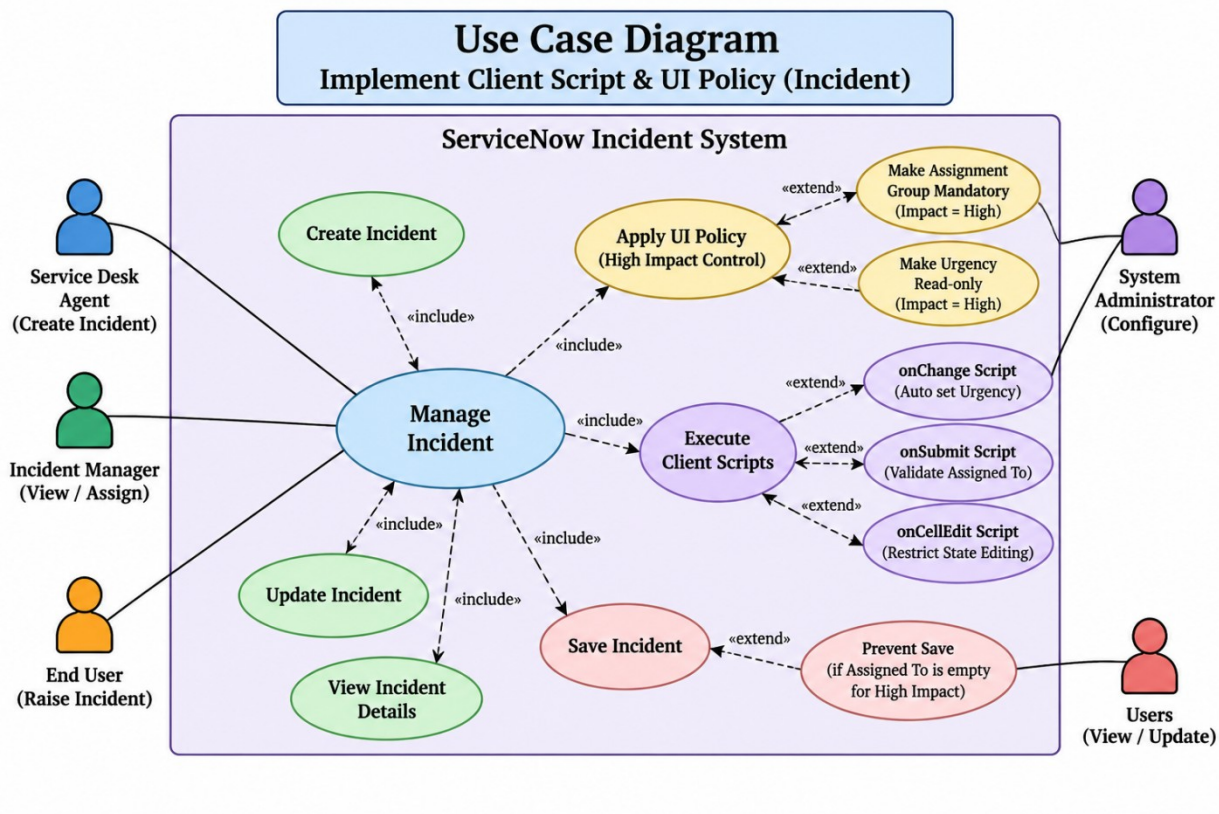


Figure 3.2: Use Case Diagram

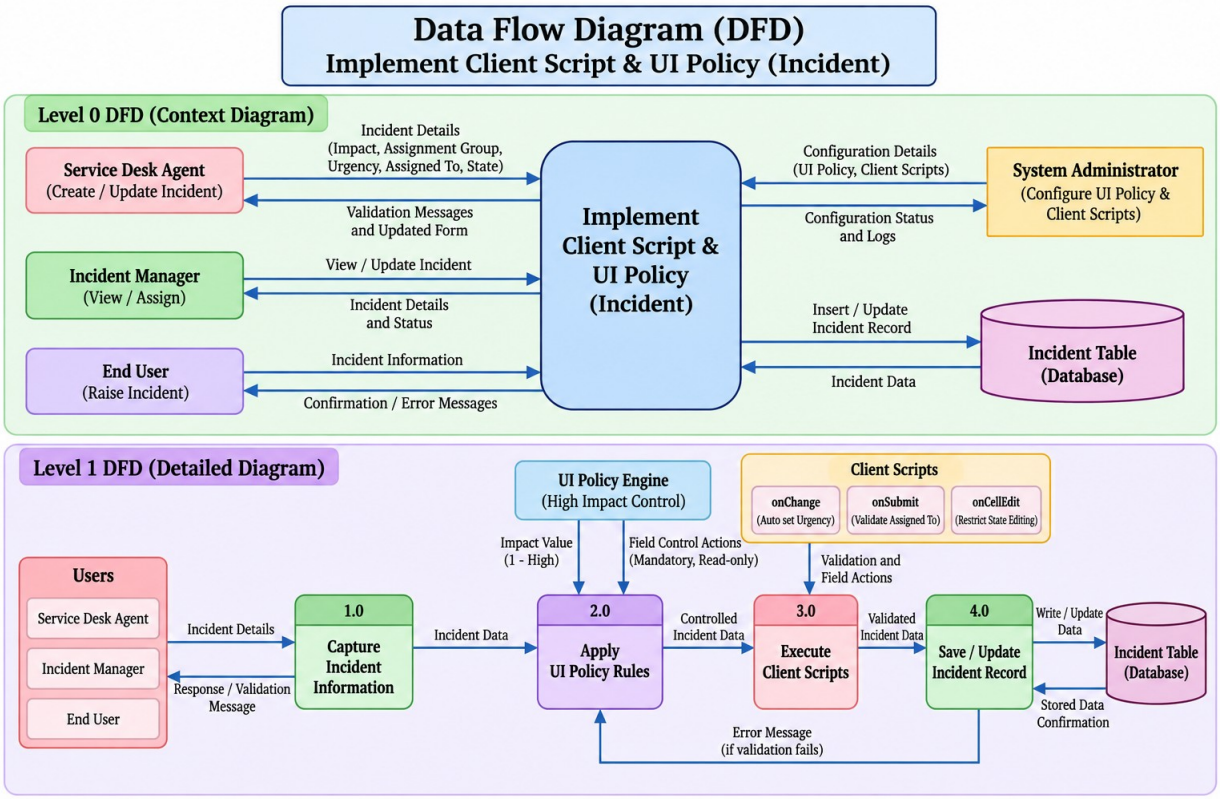


Figure 3.3: Data Flow Diagram

FINAL PROJECT REPORT

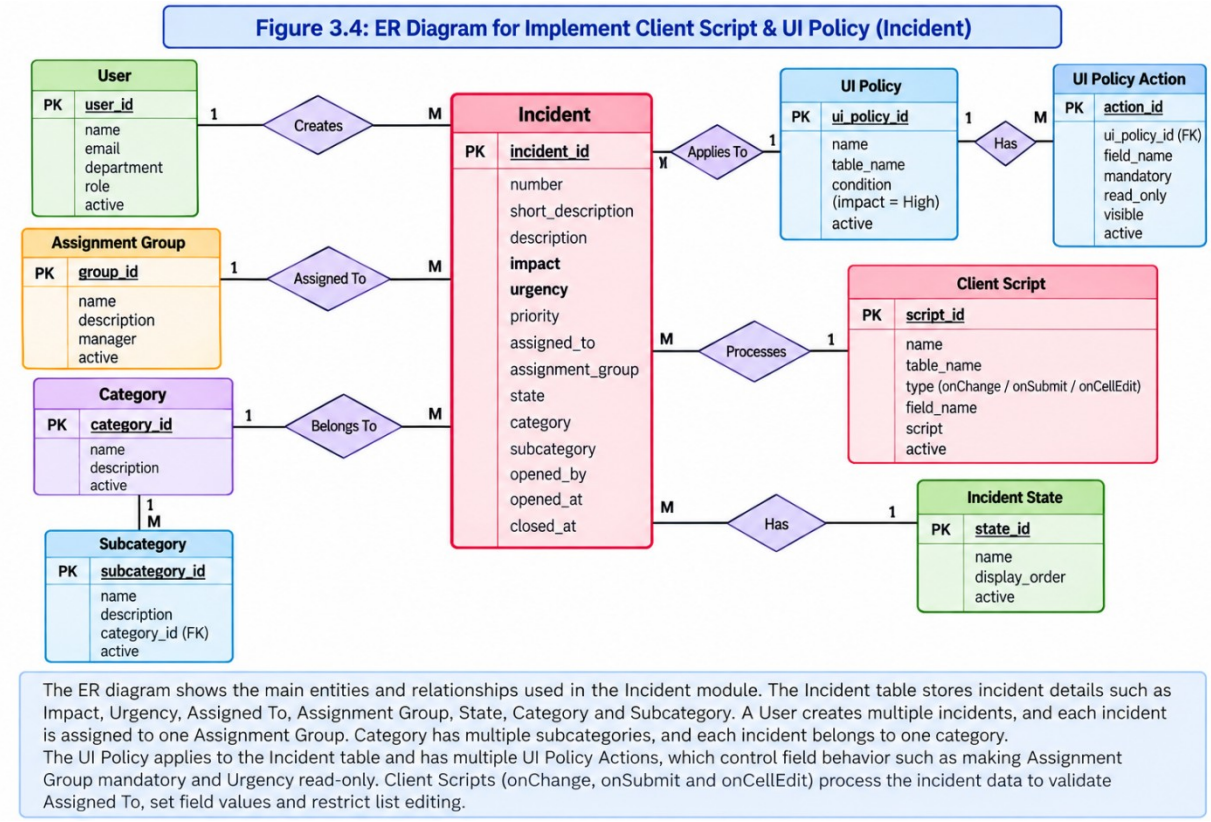


Figure 3.4: ER Diagram

Chapter 5: Development

Development

The development phase implements the proposed **Client Script and UI Policy controls for the ServiceNow Incident table**. The configuration begins with the **High Impact Control** UI Policy, which is applied when **Impact = 1 – High**. When the condition is satisfied, the Assignment Group is made mandatory and the associated UI Policy Actions control the Incident form fields. Implement Client Script & UI Po…

A UI Policy Action is then configured for the **Urgency** field. The field remains visible but becomes **read-only** when the High Impact condition is active. The configuration also supports reversing the behavior when the condition is no longer satisfied. Implement Client Script & UI Po…

The development phase also includes three Client Scripts. The **onChange Client Script** monitors the Impact field and automatically sets Urgency to High when Impact is changed to High. The **onSubmit Client Script** validates the Assigned To field and prevents submission when it is empty for a High Impact Incident. The **onCellEdit Client Script** prevents users from changing the State directly through list editing and instructs them to open the Incident form instead. Implement Client Script & UI Po… Implement Client Script & UI Po… Implement Client Script & UI Po…

Development Components

1. **High Impact Control – UI Policy**
2. **Assignment Group – Mandatory**
3. **Urgency – Read-Only UI Policy Action**

FINAL PROJECT REPORT

4. **onChange Client Script – Automatic Urgency**
5. **onSubmit Client Script – Assigned To Validation**
6. **onCellEdit Client Script – State List-Edit Restriction**

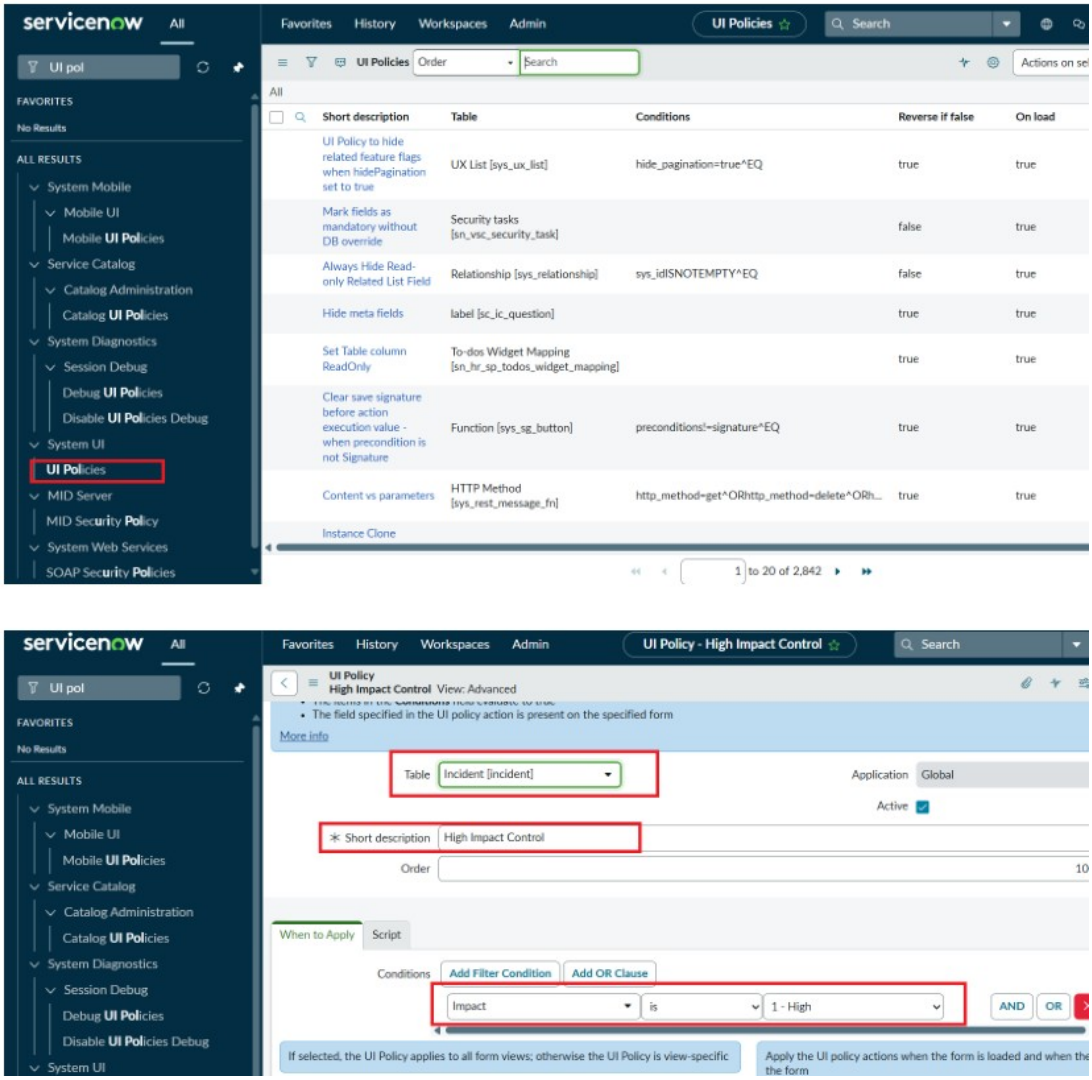


Figure 5.1: UI Policy Configuration

FINAL PROJECT REPORT

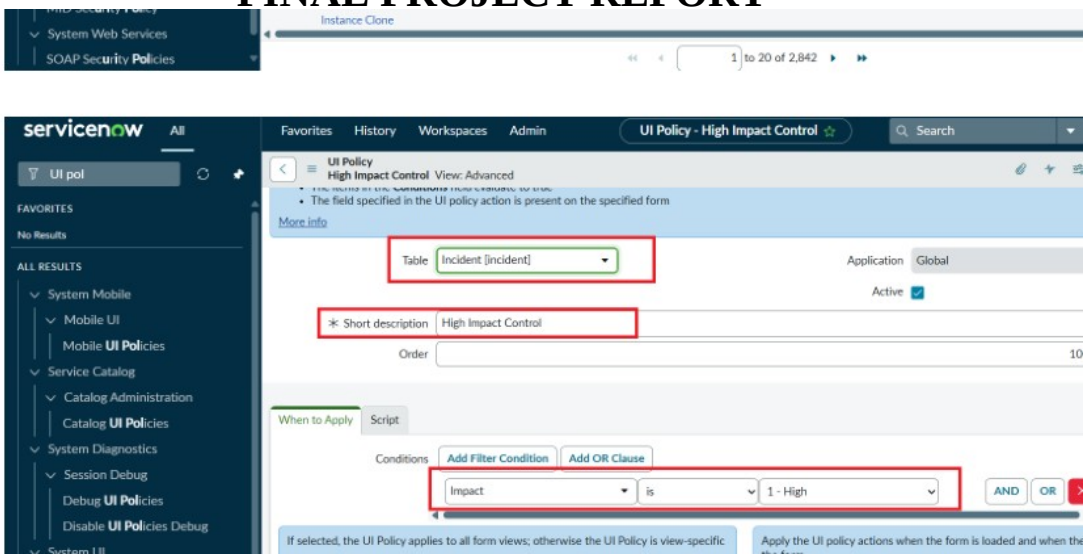


Figure 5.2: Assignment Group Configuration

Chapter 6: Testing

Testing

The testing phase verifies whether the configured **UI Policies and Client Scripts** work as intended on the ServiceNow Incident form. The testing process covers mandatory-field validation, successful Incident submission, reverse-condition behavior, list-editing restriction, and form-based State updates. Implement Client Script & UI Po…

1. Mandatory Validation Test

An Incident is created with **Impact = High** while the **Assigned To** field is left empty. When the user attempts to submit the record, the onSubmit Client Script prevents the submission and displays an error message. Implement Client Script & UI Po…

2. Successful Save Test

The Assigned To field is populated with a valid user and the Incident is submitted again. The record should save successfully without the Assigned To validation error, while the other configured behaviors continue to operate. Implement Client Script & UI Po…

3. Reverse Condition Test

The Impact value is changed from **High to Medium** to test the **Reverse if false** configuration. Assignment Group should no longer be mandatory and the Urgency read-only behavior should be reversed. Implement Client Script & UI Po…

4. List Edit Blocking Test

An attempt is made to change the **State** field directly from the Incident list. The onCellEdit Client Script displays an alert and prevents the list-based State update. Implement Client Script & UI Po…

FINAL PROJECT REPORT

5. Form-Based Update Test

The Incident is opened through the form and the State field is changed there. The update should be saved successfully, demonstrating that State changes are permitted through the Incident form rather than direct list editing. Implement Client Script & UI Po…

Testing Summary

Test Case	Expected Result
Impact = High + Assigned To empty	Submission prevented
Impact = High + Assigned To provided	Incident saved
Impact changed from High to Medium	UI Policy restrictions reversed
State edited from list	Update blocked
State edited through form	Update allowed

These tests demonstrate that the implemented configuration performs the required **validation, automation, field control, and restricted list-edit behavior** for Incident records. Implement Client Script & UI Po…

Chapter 7: Documentation and User Guidance

Documentation and User Guidance

The documentation phase provides the complete reference material required to understand, use, test, and demonstrate the **Implement Client Script & UI Policy (Incident)** project. It documents the implemented UI Policies, UI Policy Actions, Client Scripts, testing procedures, screenshots, and expected system behavior. The documentation helps users and project reviewers understand how the ServiceNow Incident configuration operates. Implement Client Script & UI Po…

1. Project Documentation

The project documentation consists of the following major components:

- **Final Project Report** – Contains the complete project details, including problem statement, ideation, requirements, design, development, testing, and conclusion.
- **README** – Provides a concise overview of the project, its objectives, implemented features, testing activities, and project structure.
- **User Manual** – Provides step-by-step instructions for creating or opening an Incident and verifying the configured UI Policies and Client Scripts.
- **Screenshots** – Provide visual evidence of configuration and testing activities.
- **Demo Video** – Demonstrates the working project and its major functionalities.

2. User Guidance

Users should first open or create an Incident and enter the required information. When **Impact is set to High**, the configured UI Policy applies the appropriate field controls. Assignment Group becomes mandatory, while the Urgency field is controlled according to the configured UI Policy Action. Implement Client Script & UI Po…

FINAL PROJECT REPORT

When Impact changes to High, the **onChange Client Script** automatically sets Urgency to High and displays an information message. Before submission, the **onSubmit Client Script** checks the Assigned To field and prevents saving when it is empty for a High Impact Incident. Implement Client Script & UI Po… Implement Client Script & UI Po…

For State changes, users should open the Incident form rather than editing State directly from the Incident list. The **onCellEdit Client Script** blocks list-based State editing and displays a message instructing the user to open the Incident. Implement Client Script & UI Po…

3. User Workflow

```
Open/Create Incident
        ↓
Enter Incident Information
        ↓
Set Impact
        ↓
Impact = High?
     ↓       ↓
    Yes      No
     ↓       ↓
UI Policy    Normal
Applied      Behavior
     ↓
Assignment Group Mandatory
     ↓
Urgency Controlled/Auto-Set
     ↓
Validate Assigned To
     ↓
Submit Incident
     ↓
Incident Saved
```

For State updates:

```
Incident List
     ↓
Attempt State List Edit
     ↓
Update Blocked
     ↓
Open Incident Form
     ↓
Change State
     ↓
Update Successfully
```

4. Documentation Outcome

The documentation and user guidance provide a structured reference for operating and reviewing the completed project. Together, the **Final Project Report, README, User Manual, screenshots, and demonstration video** document the implementation and explain how the configured UI Policies and Client Scripts support controlled and consistent Incident processing. Implement Client Script & UI Po…

FINAL PROJECT REPORT

.

Chapter 8: Demonstration

Demonstration

The demonstration phase presents the practical working of the **Implement Client Script & UI Policy (Incident)** project in ServiceNow. It shows how the configured UI Policy, UI Policy Action, and Client Scripts respond to different Incident conditions and user actions. Implement Client Script & UI Po…

1. Demonstration of High Impact UI Policy

First, an Incident record is opened or created and the **Impact** field is set to **1 – High**. The **High Impact Control** UI Policy is triggered, making the **Assignment Group** field mandatory. This demonstrates the conditional field behavior configured for High Impact Incidents. Implement Client Script & UI Po…

2. Demonstration of Urgency Control

The **Urgency** field is demonstrated after setting Impact to High. The onChange Client Script automatically sets Urgency to High, while the UI Policy Action controls the field as read-only. Implement Client Script & UI Po… Implement Client Script & UI Po…

3. Demonstration of Assigned To Validation

The Assigned To field is left empty and the user attempts to submit the Incident. The **onSubmit Client Script** detects that Impact is High and Assigned To is empty, displays the validation message, and prevents the Incident from being submitted. Implement Client Script & UI Po…

4. Demonstration of Successful Submission

A valid user is selected in the **Assigned To** field and the Incident is submitted again. The Incident should save successfully because the required validation condition has been satisfied. Implement Client Script & UI Po…

5. Demonstration of Reverse Condition

The Impact value is changed from **High to Medium**. The demonstration verifies that the UI Policy restrictions are reversed, including the mandatory behavior of Assignment Group and the read-only behavior of Urgency. Implement Client Script & UI Po…

6. Demonstration of State List-Edit Restriction

The Incident list is opened and an attempt is made to change the **State** field directly. The **onCellEdit Client Script** blocks the operation and displays a message asking the user to open the Incident form. Implement Client Script & UI Po…

FINAL PROJECT REPORT

7. Demonstration of Form-Based State Update

Finally, the Incident is opened through the form and the State field is changed there. The update is allowed and saved successfully, demonstrating the intended difference between list-based and form-based State updates. Implement Client Script & UI Po…

Demonstration Flow

```
Open/Create Incident
        ↓
Set Impact = High
        ↓
Assignment Group → Mandatory
        ↓
Urgency → Automatically High / Read-Only
        ↓
Check Assigned To
        ↓
Submit Incident
        ↓
Successful Save
        ↓
Change Impact → Medium
        ↓
Reverse UI Policy Behavior
        ↓
Test State List Editing
        ↓
List Update Blocked
        ↓
Open Incident Form
        ↓
Update State Successfully
```

Demonstration Deliverables

- **Demo_Screenshots/** – Screenshots showing the major implementation and testing steps.
- **Demo_Project.mp4** – Video demonstration of the complete project workflow.

The demonstration provides visual evidence that the configured UI Policies and Client Scripts operate according to the documented project requirements.

.Conclusion

Conclusion

The **Implement Client Script & UI Policy (Incident)** project successfully demonstrates how ServiceNow **UI Policies, UI Policy Actions, and Client Scripts** can be used together to control Incident form behavior and improve data consistency. The implementation applies conditional controls when **Impact is High**, including mandatory Assignment Group and controlled Urgency behavior. Implement Client Script & UI Po…

The project also demonstrates three Client Scripts: **onChange** automatically sets Urgency to High, **onSubmit** validates Assigned To before saving a High Impact Incident, and **onCellEdit** prevents State changes through list editing. These controls provide automated validation and controlled

FINAL PROJECT REPORT

interaction with Incident records. Implement Client Script & UI Po… Implement Client Script & UI Po… Implement Client Script & UI Po…

Testing confirms the documented behaviors, including mandatory validation, successful saving, reverse-condition functionality, list-edit blocking, and form-based State updates. Overall, the project provides a practical demonstration of using ServiceNow client-side configuration to maintain consistent Incident data and enforce defined business rules.
Conclusion

The **Implement Client Script & UI Policy (Incident)** project successfully demonstrates how ServiceNow **UI Policies, UI Policy Actions, and Client Scripts** can be used together to control Incident form behavior and improve data consistency. The implementation applies conditional controls when **Impact is High**, including mandatory Assignment Group and controlled Urgency behavior. Implement Client Script & UI Po…

The project also demonstrates three Client Scripts: **onChange** automatically sets Urgency to High, **onSubmit** validates Assigned To before saving a High Impact Incident, and **onCellEdit** prevents State changes through list editing. These controls provide automated validation and controlled interaction with Incident records. Implement Client Script & UI Po… Implement Client Script & UI Po… Implement Client Script & UI Po…

Testing confirms the documented behaviors, including mandatory validation, successful saving, reverse-condition functionality, list-edit blocking, and form-based State updates. Overall, the project provides a practical demonstration of using ServiceNow client-side configuration to maintain consistent Incident data and enforce defined business rules. Implement Client Script & UI Po…